Head of Claude Code: What Happens After Coding Is Solved

---

1. Source Info

Title: Head of Claude Code: What happens after coding is solved
Speaker: Boris Cherny (Creator & Head of Claude Code, Anthropic)
Host: Lenny Rachitsky (Lenny's Podcast)
Date: 19 February 2026
Views: 423,000+ at time of review
Link: https://www.youtube.com/watch?v=We7BZVKbCVw

---

2. TL;DR

Boris Cherny, creator of Claude Code at Anthropic, makes the case that coding (the act of writing software by hand) is now "largely solved" by AI. He has not written a single line of code by hand since November 2025, yet he ships 10 to 30 pull requests per day by running multiple AI agents simultaneously. The conversation covers how Claude Code grew from a weekend prototype to powering 4% of all public GitHub commits, the counterintuitive product principles that drove its success, why small underfunded teams build better AI products, the printing press as a historical analogy for what is happening to software, and what comes next for engineers, product managers, and designers in a world where implementation is no longer the bottleneck.

---

3. Core Thesis

Boris Cherny's central argument is that coding, defined specifically as the mechanical act of translating human intent into working software, is now a solved problem. He is careful with this claim. He does not mean that software engineering is solved, or that building great products is solved, or that the need for human judgment has disappeared. What he means is that the implementation layer (writing functions, debugging errors, handling boilerplate, running tests, iterating on pull requests) can now be reliably delegated to AI.

He offers himself as the primary evidence. Since November 2025, 100% of his code has been authored by Claude Code. He runs five or more AI agents simultaneously, each working on independent tasks across separate terminal windows. One agent runs a test suite while another refactors a module while a third drafts documentation. He monitors their progress through iTerm2 system notifications and intervenes only when an agent gets stuck or needs directional input. The result is that he ships more code in a day than most engineers ship in a week, and his role has shifted from writing code to directing intent.

The deeper layer of his thesis concerns what happens after implementation becomes cheap. Boris draws a parallel to the printing press: within 50 years of Gutenberg's invention, more printed material existed than in the previous thousand years. Costs dropped by a factor of 100. But literacy took 200 years to reach 70% of the global population, because it required infrastructure, education systems, and cultural change. Boris believes AI is doing the same thing to programming. The cost of producing software has already dropped dramatically. But the full democratization, where anyone can describe what they want in natural language and get working software, will take years to fully unfold. For practitioners right now, the implication is stark: the value of knowing how to code is declining rapidly, while the value of knowing what to build and why is increasing just as fast.

---

4. Key Insights & Frameworks

4.1 The Origin Story of Claude Code

Claude Code started as a weekend hack. Boris describes how he built the initial prototype

quickly, almost as an experiment, and the team was surprised by how intensely people responded to it. What began as a simple terminal-based tool for interacting with Claude while coding turned into one of Anthropic's most important products. The growth trajectory was steep: Claude Code went from a quick internal prototype to powering 4% of all public GitHub commits, with daily active users doubling in a single month.

What makes this origin story significant from a product perspective is that Boris didn't set out to build a massive platform. He built something he personally wanted to use, shipped it, and discovered that the demand was enormous. He describes this using the concept of latent demand: millions of engineers wanted a better way to work with AI on real code, and millions of non-engineers wanted to build software but couldn't because the barrier was too high. Claude Code didn't create this demand; it tapped into demand that was already there, waiting for the right tool to unlock it.

Boris compares this to drilling for water. The demand was underground. The product just had to reach it. For growth practitioners, this is an important framing: the most successful products in the AI era may not be ones that manufacture new behaviors, but ones that reveal and serve latent needs that were previously too expensive or difficult to address.

4.2 Why Boris Left for Cursor and Came Back in Two Weeks

One of the more revealing segments of the conversation is Boris's admission that he briefly left Anthropic for Cursor, one of the leading AI-powered code editors, and came back after just two weeks. Lenny probes this, and Boris explains that the experience actually strengthened his conviction in Anthropic's approach.

At Cursor, he got firsthand exposure to how a competitor was thinking about AI-assisted coding. But what he realized was that the underlying model quality mattered far more than the editor's user interface or feature set. He came back to Anthropic because he believed Claude's model (specifically Opus) was fundamentally better at coding tasks. He describes it this way: even though Opus is bigger and slower than Sonnet, you have to steer it less and it's better at tool use, so it ends up being faster in practice because you spend less time correcting it.

This is a significant insight for anyone evaluating AI tools. The market tends to focus on surface-level features (UI polish, keyboard shortcuts, integrations), but Boris's experience

suggests that the quality of the underlying model is the variable that actually determines productivity. The best coding tool is whichever one has the best model, full stop.

4.3 The "Coding Is Solved" Claim, With Its Qualifications

Boris's statement that "coding is largely solved" is the headline claim of the conversation, and it's worth unpacking exactly what he means and what he does not mean.

He is specific: the mechanical act of writing code (typing syntax, implementing functions, writing boilerplate, fixing bugs, writing tests) is what's solved. AI agents can now do this reliably for the vast majority of tasks. What is NOT solved is software engineering in the broader sense: deciding what to build, understanding user needs, making architectural tradeoffs, designing systems that scale, and exercising judgment about what matters. Boris sees these higher-order skills as becoming more valuable, not less, precisely because the implementation layer is now cheap.

He also acknowledges that AI coding is not perfect. Agents still get stuck. They still produce code that needs review. They sometimes go in the wrong direction and need to be redirected. Boris describes his own experience of anxiety when agents aren't working well, when they're spinning on a problem or producing low-quality output. He doesn't pretend this is a friction-free experience. But his argument is that even with these imperfections, the productivity gain is so large that it fundamentally changes how software gets built.

There's an important nuance here for growth teams: "solved" does not mean "fire your engineers." It means the bottleneck shifts. The constraint is no longer "can we build this?" but "should we build this?" and "what should we build next?" For growth engineering specifically, this means the limiting factor on experimentation is no longer implementation capacity. It's experiment design, metric selection, and strategic prioritization.

4.4 Boris's Current Coding Workflow in Detail

Boris walks through his actual day-to-day workflow in considerable detail, and this is one of the most practically valuable segments of the conversation.

He uses Claude Code exclusively in the terminal (not in an IDE). He runs five or more agent

sessions simultaneously across different terminal windows in iTerm2. Each agent is assigned a discrete, well-scoped task: a feature branch, a bug fix, a refactor, a documentation update, or a test suite. He monitors progress through iTerm2's system notifications, which alert him when an agent completes a task or hits a blocker.

His process for a typical task is: describe what he wants in natural language, let the agent work, review the output, and either approve it or give the agent additional direction. He describes shipping 10, 20, sometimes 30 pull requests in a single day, each one written entirely by Claude Code. He says he hasn't edited a single line of code by hand since November.

On top of this, he runs 5 to 10 Claude sessions in his browser on claude.ai for non-coding tasks: drafting documents, analyzing data, brainstorming ideas. His entire workday is mediated through AI agents working in parallel.

The key lesson for practitioners is that this workflow requires a different skill set than traditional coding. Instead of writing code, Boris spends his time on task decomposition (breaking large problems into agent-sized chunks), quality review (evaluating agent output for correctness and quality), and strategic direction (deciding what to build and in what order). These are fundamentally different competencies than syntax and debugging.

4.5 The Printing Press Analogy

This is the most fully developed framework in the conversation, and Boris presents it with historical specificity.

Before the printing press, Europe's literacy rate was under 1%. Within 50 years of Gutenberg's invention, more printed material was produced than in the previous thousand years. The cost of producing written material dropped by a factor of 100 over the next 50 years. But here's the critical nuance: literacy itself took roughly 200 years to reach 70% globally. The technology arrived fast, but the societal transformation was slow, because it required education systems, free time (people had to not be working on farms all day), and cultural shifts in how knowledge was valued and transmitted.

Boris then surfaces a fascinating historical detail that makes this more than a standard analogy. He references a document from the 1400s in which a scribe was interviewed about

how they felt about the printing press. Contrary to the standard narrative of displaced workers resisting new technology, the scribe was actually excited. They saw the press as liberating them from the tedious work of copying manuscripts by hand, freeing them to focus on the higher-value work of composition and original writing.

Boris explicitly identifies with this scribe. He says he has never enjoyed coding as much as he does today, precisely because he doesn't have to deal with the minutiae anymore. The mundane parts of coding (syntax, boilerplate, debugging semicolons) are gone. What remains is the creative and strategic work: architecture, product thinking, and directing AI agents toward the right outcomes.

For practitioners, the printing press analogy offers a useful time horizon. The technology revolution is happening now, but the full societal adaptation will take years, not months. People who invest in learning to work with AI tools today are positioning themselves for a transition that will unfold over a decade or more.

4.6 The Underfunding Paradox and Team Principles

Boris makes a counterintuitive argument about team structure that deserves careful attention. He argues that the best AI products come from small, deliberately underfunded teams. The Claude Code team was kept intentionally small, and Boris believes this was a key driver of its success.

His reasoning: when you give a large team generous headcount, they spend their time building processes, hiring managers, creating review cycles, and coordinating between people. When you give a small team unlimited AI tokens instead of unlimited headcount, they spend their time building product. The constraints force them to lean on AI tools for everything, which makes them better dogfooders of their own product, which directly improves the product itself. It's a virtuous cycle that only works when the team is small enough to feel the constraints.

He extends this into a specific recommendation: organizations should give every engineer unlimited access to AI coding tokens. The cost is trivial compared to engineering salaries. At Anthropic, this policy contributed to a 200% increase in per-engineer productivity.

Boris also shares three principles he tells every new person joining the Claude Code team.

First, use the product obsessively. If you're building an AI coding tool, 100% of your own coding should be done through that tool. There is no substitute for dogfooding. Second, ship fast, iterate faster. The Claude Code team ships multiple times per day. Speed is treated as a feature, not a risk. Third, stay uncomfortable. If you're not anxious about how fast things are changing, you're not paying close enough attention. The discomfort is a signal that you're working at the frontier.

4.7 Which Roles Will AI Transform Next

Boris spends a meaningful portion of the conversation discussing which professional roles will be affected by AI and in what order. He is direct about his predictions.

He believes the title "software engineer" will start to go away, replaced by "builder." This isn't just semantic; it reflects a real shift in what the job entails. A builder is someone who creates things using whatever tools are available, rather than someone who writes code in a specific language. The identity shift matters because it changes what people optimize for: outcomes rather than technical process.

He extends this to product managers and designers. On the Claude Code team, everyone codes, including the product manager, the engineering manager, the designer, the finance person, and the data scientist. AI makes this possible because you don't need deep expertise in a domain to direct an AI agent effectively; you need enough understanding to evaluate the output and give useful direction.

Boris also discusses a poll Lenny references about which roles are most enjoying working with AI. The data suggests that people who are embracing AI tools and crossing disciplinary boundaries are finding their work more rewarding, while those who are clinging to narrow specializations are feeling more threatened. Boris's advice is clear: be a generalist. The era of the narrow specialist is ending.

4.8 How Cowork Was Built in 10 Days

Boris describes Cowork (Anthropic's AI assistant that can operate a web browser autonomously) and reveals that it was built in just 10 days. This anecdote serves as a concrete illustration of several themes in the conversation.

First, it demonstrates the speed that's possible when a small team uses AI tools intensively. Ten days from concept to a working product that can open Chrome, navigate web pages, fill out forms, and complete multi-step workflows autonomously. Boris mentions an example of an Anthropic employee who went on parental leave and used Cowork to fill out medical forms automatically: it loaded the browser, logged in, filled out the PDFs, and submitted them.

Second, it illustrates the latent demand principle again. The team tried a similar experiment a year earlier and it didn't work because the model wasn't ready. But now the model quality had crossed a threshold where browser automation actually worked reliably. The demand was there all along; the technology just had to catch up.

4.9 The Three Layers of AI Safety at Anthropic

Boris discusses Anthropic's approach to AI safety with genuine depth, which is notable given that safety is sometimes treated as an afterthought in product-focused conversations. He describes three distinct layers.

The first layer is model-level safety: the training and alignment work that ensures the base model behaves responsibly. The second layer is product-level safety: the guardrails, filters, and controls built into products like Claude Code that prevent misuse. The third layer is organizational-level safety: the culture, processes, and governance structures at Anthropic that ensure safety considerations are integrated into every decision.

Boris frames this as non-negotiable. He doesn't present safety as a tradeoff with capability or speed; he presents it as a prerequisite for building products that people can trust and that companies can deploy at scale.

4.10 Anxiety at the Frontier

One of the most human moments in the conversation is Boris's candid discussion of anxiety. He doesn't project the confident calm that most tech leaders default to when discussing AI. Instead, he acknowledges that working at the frontier of AI is genuinely anxiety-inducing.

He describes the specific experience of anxiety when AI agents aren't working well: when

they're spinning, producing bad output, or going in the wrong direction. He also describes a broader existential anxiety about the pace of change and the implications for people's careers and livelihoods.

His advice on this is striking: if you're not anxious, you're not paying attention. He normalizes the discomfort rather than dismissing it. And he suggests that the anxiety itself can be productive if channeled into action: experimenting with tools, learning new skills, and staying on the frontier rather than retreating from it.

4.11 Boris's Ukrainian Roots and Post-AGI Plans

Toward the end of the conversation, Boris discusses his Ukrainian background and how it has shaped his perspective. He connects the resilience and resourcefulness required by his upbringing to the mindset needed to work in a field that is changing as rapidly as AI.

When asked about his post-AGI plans (what he would do if artificial general intelligence were achieved), Boris says he's interested in education. This is consistent with the printing press analogy: if AI is democratizing the ability to build software, the bottleneck becomes education, helping people understand what's possible and how to direct these tools effectively. It's a long-term, systems-level view that goes beyond the immediate product work.

4.12 Advice for Building AI Products

In the final sections of the conversation, Boris offers extended tactical advice for anyone building AI products. He recommends using the best available model, even if it's more expensive or slower, because the reduction in steering and correction time more than compensates. He emphasizes the importance of experimentation: try things, ship them, see what works, and iterate quickly. He warns against over-planning and under-shipping.

He also discusses his thoughts on OpenAI's Codex, acknowledging it as a competitor but expressing confidence that model quality will remain the decisive factor, and that Anthropic's models are ahead on the dimensions that matter most for coding: tool use, instruction following, and the ability to work autonomously on complex multi-step tasks.

5. Tactical Playbook

5.1 Set Up a Multi-Agent Workflow

Boris runs five or more Claude Code agents simultaneously, each on an independent task. He monitors them through terminal notifications and intervenes only when needed.

What to do: Set up your terminal to support parallel agent sessions. Start with two agents on independent tasks and scale up as you build comfort with multi-stream supervision. The skill you're developing is task decomposition: breaking large problems into agent-sized chunks that can run in parallel.

5.2 Give Your Team Unlimited AI Tokens

At Anthropic, giving engineers unlimited tokens contributed to a 200% increase in per-engineer productivity. The cost of tokens is trivial compared to engineering compensation.

What to do: Calculate the annual cost of unlimited AI tokens for your growth engineering team. Compare it to the cost of one additional engineer. Present this to leadership as a headcount-equivalent investment that ships immediately with zero ramp time.

5.3 Redefine Your Experiment Backlog

When implementation cost drops by 5 to 10x, experiments that were previously too expensive to justify become viable. The constraint shifts from "can we build this?" to "should we build this?"

What to do: Review your current experiment backlog. Identify every experiment that was deprioritized because of implementation complexity. Reassess each one under the assumption

that implementation cost is 10x lower than when it was originally estimated. Reprioritize based on expected impact, not expected effort.

5.4 Dogfood Relentlessly

Boris's first principle for new team members is to use the product obsessively. For growth teams, this means using your own product the way a new user would, regularly and critically.

What to do: Schedule a weekly 30-minute session where the entire growth team uses the product from a new-user perspective. Document friction points, confusing flows, and moments of delight. Feed these directly into your experiment pipeline.

5.5 Frame Work as Outcomes, Not Implementation

Boris predicts the "software engineer" title will evolve into "builder." The identity shift matters because it changes what people optimize for.

What to do: In your next sprint planning, frame every ticket in terms of the outcome it produces ("increase activation rate by 5%") rather than the implementation it requires ("add tooltip to onboarding step 3"). This aligns the team's attention with value creation rather than task completion.

5.6 Use AI on Real Production Work, Not Toy Projects

Boris is emphatic that the only way to build genuine intuition for AI-assisted development is to use it on real work. The gap between demo-quality output and production-quality output is where all the real learning happens.

What to do: Pick one real production task this week and commit to completing it entirely through AI-assisted coding. Document what worked, what failed, and where you had to intervene. Share the learnings with your team.

5.7 Invest in Review Infrastructure Before Scaling Velocity

The research data shows that teams with high AI adoption merge 98% more pull requests, but PR review time increases 91%. If you scale code generation without scaling review capacity, you'll create a bottleneck.

What to do: Before pushing your team to generate more code with AI, invest in automated testing, CI/CD guardrails, linting, and observability. These are the systems that let you safely absorb higher velocity without sacrificing quality.

---

6. Growth Engineering Lens

6.1 Experimentation Velocity as a Competitive Advantage

The most immediate implication for growth engineering teams is a step-function increase in experimentation velocity. If implementation cost drops by 5 to 10x, the number of experiments you can run per quarter increases proportionally. This is not incremental; it changes the fundamental economics of experimentation.

Boris's own workflow demonstrates the scale: shipping 10 to 30 PRs per day is an order of magnitude above what most senior engineers accomplish. Applied to a growth engineering team, this means dozens of concurrent A/B tests, feature variants built and torn down in hours rather than weeks, and growth loops iterated with unprecedented speed.

However, the METR study provides a critical counterpoint. Their randomized controlled trial found that developers using AI tools actually took 19% longer on certain tasks. The key variable is task type. AI excels at well-defined implementation tasks (building experiment variants, creating analytics pipelines, writing test code) but struggles with novel architectural decisions. Growth engineers should match the tool to the task accordingly.

6.2 Infrastructure and Data Pipelines

Growth engineering depends heavily on data infrastructure: event tracking, experimentation platforms, feature flagging systems, analytics dashboards. These are exactly the kinds of well-defined, pattern-heavy systems where AI coding excels. A growth engineer directing an AI agent to build an event tracking pipeline or set up a new experiment in a feature flagging system can accomplish in an hour what previously took days.

6.3 The PR Review Bottleneck

Teams with high AI adoption merge 98% more pull requests, but PR review time increases 91%. This reveals that human review is becoming the bottleneck when code generation is cheap. Growth engineering teams need to either automate reviews where safe to do so (linting, type checking, test coverage gates), parallelize reviews across team members, or fundamentally rethink what requires human review versus what can be validated automatically.

6.4 Code Quality as a Growth Risk

AI-assisted code can increase issue counts approximately 1.7x if not paired with governance. For growth teams specifically, buggy experiments pollute data, create negative user experiences, and erode organizational trust in the experimentation platform. The tactical response is to invest in automated testing, CI/CD guardrails, and observability *before* scaling AI-assisted experiment velocity. The infrastructure has to be in place first.

---

7. Growth Marketing Lens

7.1 The Democratization of Software Creation

Boris's printing press analogy has direct implications for growth marketing. If AI makes software creation accessible to non-engineers, the addressable market for developer tools, no-code platforms, and AI coding assistants expands dramatically. Growth marketers should be thinking about acquisition strategies that target "aspiring builders": people with ideas who lack traditional coding skills.

Elena Verna's recent work at Lovable (which hit $200M ARR in one year with 100 employees) demonstrates this in practice. Their most powerful growth strategy has been giving the product away for free, because the product itself generates word-of-mouth among people who couldn't previously build software. The product is the marketing.

7.2 Content and Thought Leadership as Acquisition

Boris's appearance on Lenny's Podcast is itself a growth marketing case study. The episode generated 423,000+ views and triggered viral coverage across VentureBeat, Fortune, TechCrunch, and tech Twitter. For growth marketers at AI companies, executive thought leadership on high-reach podcasts and newsletters is a high-leverage acquisition channel that generates both awareness and credibility simultaneously.

7.3 Channel Disruption

Elena Verna's research shows that AI is actively dismantling traditional distribution channels. Web traffic is declining as users turn to chatbots and AI assistants for answers rather than search engines or websites. Growth marketers need to adapt their channel strategies: SEO may become less effective while AI-native distribution (being recommended by AI assistants, building AI-integrated products, optimizing for conversational discovery) becomes more important.

7.4 Retention Through Innovation Speed

Spotify's case study is instructive. Their CEO stated that their best developers haven't written

a line of code since December 2025, and the result was shipping 50+ new features throughout the year. The retention loop is clear: faster engineering velocity leads to more features, more features deliver more user value, more value drives higher retention. Growth marketers should connect engineering velocity metrics to retention metrics and communicate feature velocity as a competitive advantage in positioning.

7.5 The "Time to Wow" Metric

When AI tools can build a functional prototype in minutes, the "time to wow" for new users drops dramatically. Growth marketers should optimize for this metric explicitly: how quickly can a new user experience the core value proposition? If your product leverages AI to deliver results fast, your activation metrics and onboarding flows should be designed to showcase this speed advantage within the first session.

---

8. Contrarian / Non-Obvious Takes

8.1 The Scribe Was Excited

The most counterintuitive moment in the conversation is the historical detail about scribes and the printing press. The standard narrative is that technology displaces workers who resist change. But the historical record shows that skilled scribes were actually excited about the press because it freed them from copying. Boris identifies with this: he says he's never enjoyed coding more than he does now, precisely because the tedious parts are gone. The contrarian take for practitioners: the best engineers will embrace AI coding because it eliminates the parts of the job they never liked.

8.2 Underfunding as a Feature

Conventional wisdom says important projects should be generously resourced. Boris argues the opposite for AI products. Small teams with limited headcount but unlimited AI tokens produce better products because the constraints force them to use their own tools intensively, which improves the product through better dogfooding. This is a genuine inversion of standard scaling logic.

8.3 The Model Matters More Than the Tool

While the market obsesses over editor features (Cursor's UI, Windsurf's experience, Copilot's integrations), Boris makes the case that the underlying model quality is the decisive factor. He uses Opus despite it being slower and more expensive than Sonnet because it requires less steering and produces better results per interaction. The contrarian take: the "best coding tool" is whichever one has the best model, regardless of UX.

8.4 Anxiety Is the Correct Response

Most tech leaders project calm confidence. Boris openly acknowledges that working at the frontier is anxiety-inducing, and he suggests that the absence of anxiety means you're not paying attention. This is a genuinely non-obvious position in an industry that rewards optimism and certainty.

---

9. Quotable Moments

1. **"100% of my code is written by Claude Code"** : the opening statement that sets the scale of the shift. Not 80%, not most. All of it.
2. **"Coding is largely solved"** : the thesis in four words. Provocative, but carefully qualified throughout the conversation.
3. **"Give engineers unlimited tokens"** : a specific, implementable recommendation that challenges how organizations budget for AI tools.

4. **"The scribe was actually excited"** : the historical detail that reframes the displacement narrative entirely.
5. **"In a year or two, it's not going to matter"** : on whether people should learn to code. The starkest prediction in the conversation.

---

10. What to Revisit

- **(00:00 to 03:45) The opening montage** : Boris's workflow described in his own words. Watch this to internalize what "100% AI-written code" looks like in practice.
- **(03:45 to 05:35) Why he left for Cursor and came back** : the competitive insight about model quality versus editor features.
- **(08:41 to 13:29) The origin story of Claude Code** : how a weekend hack became 4% of GitHub commits. Essential for understanding product-market fit and latent demand.
- **(16:17 to 17:32) Boris's current workflow** : the specific details of running 5+ agents simultaneously. Model your own setup after this.
- **(24:02 to 26:48) Principles for the Claude Code team** : the three principles Boris gives every new team member.
- **(26:48 to 27:55) Why unlimited tokens matter** : the business case for giving engineers unlimited AI compute. Share this section with your VP of Engineering.
- **(32:15 to 36:01) The printing press analogy** : the best-articulated framework in the conversation. Revisit when explaining AI's long-term impact to non-technical stakeholders.
- **(46:32 to 51:53) Latent demand and Cowork** : how Anthropic identified and validated unmet demand. Essential product strategy material.
- **(54:04 to 59:35) AI safety at Anthropic** : the three layers framework. Useful context for anyone deploying AI tools in production.
- **(1:03:21 to 1:08:38) Advice for AI products and Claude Code tips** : concentrated tactical advice. Revisit before starting any AI-assisted project.

---

11. Sources & Further Reading

1. **Gergely Orosz, "Building Claude Code with Boris Cherny" (The Pragmatic Engineer)**

https://newsletter.pragmaticengineer.com/p/building-claude-code-with-boris-cherny Deep technical dive into Claude Code's architecture and Boris's engineering philosophy.

2. **Lenny Rachitsky's episode page and transcript**

https://www.lennysnewsletter.com/p/head-of-claude-code-what-happens Full episode transcript and Lenny's own takeaways.

3. **TechCrunch: Spotify's best developers haven't written code since December**

https://techcrunch.com/2026/02/12/spotify-says-its-best-developers-havent-written-a-line-of-code-since-december Enterprise case study validating Boris's claims about AI-written code at scale.

4. **Fortune: Claude Code gives Anthropic its viral moment**

https://fortune.com/2026/01/24/anthropic-boris-cherny-claude-code-non-coders-software-engineers/ Business context on Claude Code's market impact and growth trajectory.

5. **METR: Measuring the Impact of Early-2025 AI on Developer Productivity**

https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/ Rigorous RCT showing AI tools made experienced developers 19% slower on certain tasks. Essential counterbalance to productivity optimism.

6. **Elena Verna on Lenny's Podcast: The new AI growth playbook for 2026**

https://www.lennysnewsletter.com/p/the-new-ai-growth-playbook-for-2026-elena-verna Elena's framework for how growth loops, distribution, and PLG are changing in the AI era.

7. **2025 DORA Report on AI and Software Engineering Performance (InfoQ)**

https://www.infoq.com/news/2026/03/ai-dora-report/ Industry benchmark data on how AI adoption correlates with engineering delivery metrics.

8. **Waydev: 8 Game-Changing Insights from Anthropic's Boris Cherny**

https://waydev.co/8-game-changing-insights-from-anthropic-claudecode-boris-cherny/ Structured summary of Boris's key claims with supporting data.

9. **Boris Cherny's personal site and Twitter (@bcherny)**

https://borischerny.com / https://x.com/bcherny His viral thread on his Claude Code setup and ongoing commentary.

10. **Faros AI: The AI Productivity Paradox Research Report**

https://www.faros.ai/blog/ai-software-engineering Data on the gap between individual AI productivity gains and organizational delivery velocity.